DataDriven

What to learn, in what order, and what to skip. Written from the interviews that actually decide hiring outcomes, not from the job descriptions that list every tool the company has ever touched.

Every list of data engineering skills you'll find online is too long. Most working data engineers use four things daily: SQL, Python, a warehouse, and an orchestrator. The remaining tools on the famous Awesome-Data-Engineering lists are real but optional, and the order you learn them matters more than the count. This is the order, written from the interviews that actually decide hiring outcomes rather than the JDs that list every tool the company has ever touched.

If you can write a window function from memory and parse a malformed JSON file in Python, you're past stage one. If you've done either with a deadline on the line, you're past stage two. Most people who think they need this roadmap need stage three.

Each stage has a thing to learn and a thing to ship. Skip neither. The 'ship' part is what turns reading into recall.

01 SQL to fluency

SELECT, JOIN, GROUP BY with HAVING, window functions, CTEs, recursive CTEs, NULL handling, conditional aggregation with FILTER, the difference between COUNT(col) and

DataDriven

drop rows.

- Don't read about SQL. Write SQL against a Postgres database and let the wrong answers tell you what you don't know.
- Ship: one analysis project that touches a real dataset and ends with a query you can defend out loud.

02 Python the way a data engineer writes it

Pandas for groupby/merge/pivot, the standard library for file parsing (CSV, JSON, gzipped logs, fields-of-fields), enough OOP to write a class with three methods and not embarrass yourself, and the kind of error handling that distinguishes a script from a job. Skip LeetCode-style algorithms; they don't show up in the rounds you care about.

- If pandas feels slow, you're holding it wrong. Learn vectorization before you learn Polars.
- Ship: a script that ingests a messy CSV, validates it, and writes to a warehouse table.

03 Dimensional modeling

Star schema, snowflake, the difference, when to denormalize on purpose. Slowly changing dimensions Type 1 versus Type 2 versus Type 6, and which one a real product actually needs. Grain. Always grain. State the grain before you draw the table. The single biggest separator between mid-level and senior data engineers is whether dimensional thinking is automatic or effortful.

- Read Kimball's Data Warehouse Toolkit. There is no shortcut. The book is forty years old and still right.
- Ship: a five-table dimensional model for a product you understand (your gym, a side project, a hobby) with the grain stated for every fact.

04 One warehouse, deeply

Pick one of Snowflake, BigQuery, or Postgres. Learn it past surface depth: query planning, partitioning, clustering, materialized views, the dialect quirks that change which queries are cheap. Going wide across all three is what bootcamps teach. Going deep on one is what gets you hired and lets you contribute on day one.

- BigQuery if you're targeting Google or analytics-heavy startups. Snowflake for most mid-market. Postgres for working knowledge that translates everywhere.

05 Orchestration and the failure modes that come with it

DataDriven

schema drift. The phrase 'exactly-once' and why it's almost always actually 'at-least-once with deduplication.' This is the content of the system design round.

- Dagster and Prefect are real. Airflow is what you'll interview on. Learn both eventually; learn Airflow first.
- Ship: a DAG that ingests something on a schedule, handles a deliberate failure, and recovers without manual intervention.

06 Interview practice

By now you know the material. What you don't have yet is the speed and the calm. Time-box SQL problems at 20 minutes, Python at 30, modeling at 45, design at 60. Do at least three full mock loops out loud before any onsite. The gap between knowing the answer and saying the answer is closed only by repetition.

- Use the round-by-round guides on this site for company-specific patterns.
- Ship: an offer. The point of the roadmap is the offer, not the roadmap.

Overrated. Spark before SQL. Five cloud platforms instead of one. Kafka unless you're interviewing somewhere that actually runs on Kafka. Building a portfolio project in month one before you can write the queries the project depends on. Memorizing Airflow operators. Watching YouTube videos at 1.5x speed. Reading about a tool instead of running it. dbt before you understand what a fact table is.

Underrated. Writing the same query three different ways and benchmarking them. Reading other people's code in production warehouses. Doing the boring schema-drawing exercise out loud. Asking working engineers what they actually do on Tuesdays, which is rarely what the job description says. Pairing with someone on a real bug. The first three bullets scale; the rest compound.

DataDriven

S3 or GCS, what they're for, what an object store does that a filesystem doesn't. IAM at the level of 'I can reason about a role and a policy.' One compute primitive (Lambda or Cloud Functions). One pipeline service (Step Functions, Dataflow, Glue). That's the surface area. Nobody is going to ask you to write Terraform on a whiteboard.

The cloud platform you pick should match the company you're targeting. AWS for breadth, GCP for analytics shops, Azure for enterprise. If you have no preference, AWS has the most jobs and translates into the others.

Spark unless you're targeting Netflix, Databricks, Airbnb, or a large adtech shop. Kafka if you're not interviewing somewhere that genuinely needs sub-second latency. Flink at all, unless you're applying to one of the five companies in the industry that ask Flink questions. dbt unless the company is dbt-native. Terraform, Kubernetes, Helm, ArgoCD: these are platform tooling, not data engineering, and they show up in the loop maybe one time in fifty.

If a job listing names all of these, the listing was written by a recruiter who read another listing. The work itself is closer to SQL and Python than to the tool inventory.

Sooner than you think. Most engineers wait until they feel ready, which is never. The signal you're ready to interview is not "I know everything" but "I can pass a single SQL round without freezing." Once that's true, interview while you continue to study; rejected loops

DataDriven

For the round-specific deep dives, start with the SQL round walkthrough and the modeling round walkthrough. For the full loop, see the interview prep pillar.

How long does this take if I'm starting from scratch? +

Can I become a data engineer without a CS degree? +

Should I get a certification? +

What about a portfolio project? +

What's the fastest path if I'm already a software engineer? +

02 / WHY PRACTICE

Stage 1 is SQL. Start there.

DataDriven

a top-tier study technique, while re-reading and highlighting rank near the bottom

02

76% of hiring managers reject on the coding task, not the resume

From HackerRank's 2024 Developer Skills Report. Candidates who look strong on paper still fail the live screen if they haven't done timed, executable practice

03

Five problem shapes cover 80% of data engineer loops

Dedup, sessionization, top-N-per-group, slowly-changing dimensions, partition tricks. Writing the shapes by hand turns the unfamiliar into pattern recognition

Open a SQL problem >>>

INTERVIEW PREP

→

50+ guides covering every round, company, and role

INTERVIEW ROUNDS

- SQL Round
- Python Round
- System Design
- Data Modeling
- Behavioral

BY COMPANY

- Stripe
- Airbnb
- Netflix
- Uber
- Databricks

BY ROLE

- Senior DE
- Staff DE
- Analytics Engineer
- ML Data Engineer
- Junior DE

QUESTION SETS

- Top 100 Questions
- FAANG Questions
- SQL Questions
- 50 Questions
- Take-Home Examples

DataDriven

Product

Lessons

Practice Problems

Mock Interview

Daily Challenge

Discuss

Download iOS App

Resources

Data Engineer Interview Prep

SQL Interview Questions

All Resources

Blog

About

Contact

Pipeline architecture tips, weekly

SQL patterns, Python tricks, and pipeline architecture insights for your next interview. Sent every Wednesday.

you@company.com Subscribe

Terms Privacy Security Help